

DEPARTMENT OF COMPUTER AND SYSTEMS ENGINEERING
FACULTY OF ENGINEERING — AIN SHAMS UNIVERSITY



COURSE PROJECT
CSE 471 - CSE 472: ARTIFICIAL INTELLIGENCE

PROJECT REPORT
QUORIDOR GAME REPORT

PROJECT LINKS

[GITHUB REPOSITORY](#) • [VIDEO DEMONSTRATION](#)

PROJECT CONTRIBUTORS

CONTRIBUTOR NAME	STUDENT ID	BYLAW
KEROLOS NADY AZMY	2300375	2023
MINA HANY EID	2300434	2023
MARK SAMEH MAHER	2301079	2023
YOUSSEF KAMEL IBRAHIM	2300651	2023
MICHEAL EHAB TAWFIK	2300916	2023
ARSANY HANY ANWAR	2200655	2018

SPRING 2026

Contents

1	Introduction	1
2	Design Decisions and Architecture	2
2.1	Overall code organisation	2
2.2	Game logic design	2
2.3	User interface design	2
2.4	AI implementation	3
2.5	Application state machine	3
2.6	Graphics and user experience	3
3	Assumptions Made During Development	4
4	Implementation Challenges	5
4.1	AI performance	5
4.2	AI wall placement blocking itself	5
5	AI Algorithm Explanation	6
5.1	Core search: Minimax with Alpha-Beta pruning	6
5.2	Evaluation function	6
5.3	Move ordering for better pruning	6
5.4	Transposition table	7
5.5	Pathfinding for distance estimation	7
5.6	Wall placement strategy	7
5.7	Difficulty levels configuration	7
5.8	AI turn implementation	8
5.9	Performance considerations	8
	References	9

1 Introduction

Quoridor is an abstract strategy board game known for its simple rules yet deep tactical complexity. Invented by Mirko Marchesi in 1997, the game is played on a 9×9 grid where two players each control a pawn and ten walls. The objective is to be the first to reach the opponent's starting side. On each turn, a player either moves their pawn orthogonally (with special jump rules when facing the opponent) or places a wall to block the opponent's progress, provided that the wall does not completely trap any player.

The problem addressed by this project is the complete implementation of Quoridor as a digital game with a graphical user interface, support for both human vs. human and human vs. computer modes, and an AI opponent with adjustable difficulty levels. This requires correctly enforcing all game rules, implementing efficient pathfinding for wall-placement validation, and designing an AI that can make intelligent decisions using game-tree search and heuristic evaluation.

The primary objectives of the project are:

- Develop a fully functional, visually appealing Quoridor game using Python and Pygame.
- Provide a clear and intuitive user interface with move hints, wall previews, and turn indicators.
- Implement a computer opponent that uses minimax search with alpha-beta pruning, configurable depth, and a custom evaluation function.
- Offer three distinct difficulty levels (Easy, Medium, Hard) that scale search depth, heuristic weights, and strategic wall placement.
- Include bonus features such as undo/redo functionality to enhance the user experience.

The scope of this report covers the design decisions and architecture, assumptions made during development, implementation challenges and solutions, a detailed explanation of the AI algorithm, and a description of the bonus features. The complete source code, a demo video, and installation instructions are available in the accompanying GitHub repository.

2 Design Decisions and Architecture

The Quoridor implementation follows a modular, model-view-controller (MVC) inspired architecture to separate game logic, user interface, and artificial intelligence. This design improves maintainability, testability, and clarity.

2.1 Overall code organisation

The source is split into three top-level packages:

- game/** Core game model: board state, players, walls, movement and wall placement rules, pathfinding for validation.
- ui/** View and controller: Pygame-based rendering, input handling, menus, and visual feedback.
- ai/** AI opponent: minimax search, evaluation functions, move ordering, transposition table, difficulty configuration.

The entry point `main.py` manages the application state machine (main menu, difficulty selection, playing) and the main loop.

2.2 Game logic design

- **Board class (`game/board.py`):** Stores walls in a list and a set for fast look-up. Provides `can_step()` to check if a single orthogonal move is blocked by a wall, using the board's wall set. Delegates move generation and wall validation to separate rule functions.
- **Player class (`game/player.py`):** Simple data container with position, walls left, and a computed `goal_row` property (0 for player 1, 8 for player 2).
- **Rule functions (`game/rules.py`):** Stateless implementations of movement generation (including jumping and diagonal jumps) and wall placement validation (bounds, overlap, crossing, and path existence for both players). Path existence is verified with BFS.
- **Game class (`game/game.py`):** Orchestrates turns, applies validated actions, detects win conditions, and manages AI turns. It holds the board, players, turn index, and game mode (PvP or AI). AI moves are called through `do_ai_move()` after a configurable delay.

2.3 User interface design

- **Renderer class (`ui/renderer.py`):** A stateless painter that draws the board, pawns, walls, hover previews, move hints, side panel, and menus. It uses a transparent overlay for hints and semi-transparent goal rows. Two additional methods draw the main menu and difficulty selection screen.
- **InputHandler class (`ui/input_handler.py`):** Maintains UI state (mode: move/wall_h/wall_v, selected pawn, valid moves, hovered cell or wall). It converts mouse positions to cell or wall coordinates, handles clicks and keyboard shortcuts, and delegates actions to the Game instance. The handler blocks interaction during the AI's turn.

- **Constants (ui/constants.py):** Centralises all geometry (cell size, gap, board padding), colours, and initial wall count. This makes it easy to tweak the visuals without touching logic.

2.4 AI implementation

- Difficulty levels (ai/config.py): Three modes (Easy, Medium, Hard) defined by search depth, evaluation weights, and use of A* for distance estimation.
- Minimax with alpha-beta pruning (ai/search.py): Recursive search that maximises the AI's evaluation score.
- Evaluation function (ai/evaluation.py): Computes a score from three terms – distance to goal, remaining walls, and mobility. The AI tries to minimize its own distance, maximize the opponent's distance, and keep a wall/mobility advantage.
- Move ordering (ai/move_ordering.py): Sorts pawn moves heuristically to improve alpha-beta pruning.
- Wall placement strategy (ai/wall_strategy.py): For Hard difficulty, the AI also considers candidate walls near both players' positions. Walls that significantly block the opponent while preserving the AI's own path receive high scores.
- AI controller (ai/ai_controller.py): Compares the best pawn move score against the best wall move score (Hard only) and returns the chosen action.

2.5 Application state machine

main.py implements a simple state machine with three states:

- `main_menu`: shows Human vs Human or Human vs AI options.
- `difficulty_menu`: lets the player choose Easy/Medium/Hard difficulties
- `playing`: runs the game loop, handles AI delay, and calls the renderer with current UI state.

This separation keeps the main loop short and easy to extend.

2.6 Graphics and user experience

- Pygame is used for cross-platform 2D rendering.
- All UI elements use a dark theme with accent colours for each player (red / cyan).
- Hover effects on cells, walls, and buttons provide immediate feedback.
- Move hints are drawn as semi-transparent green rectangles; jump moves are indicated in gold.
- A side panel displays player names, wall counts, current mode, and a control legend.
- AI moves are delayed by 500ms to make the AI's thinking visible to the human player.
- Win overlay dims the board and shows a congratulatory message.

3 Assumptions Made During Development

While implementing the Quoridor rules and AI, the following assumptions were made to simplify development:

1. Game rules are assumed to be like the original game
2. Two-player version only as specified in the requirements.
3. Wall count Each player begins with 10 walls, as per standard 2-player Quoridor.
4. Wall placement anchors
 - A horizontal wall is placed below a cell row, spanning two columns. Its anchor (r, c) means the wall occupies the gap between rows r and r+1, columns c and c+1.
 - A vertical wall is placed to the right of a cell column, spanning two rows. Its anchor (r, c) occupies the gap between columns c and c+1, rows r and r+1.
5. Path existence after wall placement: The game uses a BFS from each player's current position to their goal row. If placing a wall makes either player's path impossible, the placement is rejected. The BFS considers only orthogonal moves and respects existing walls.
6. AI difficulty levels
 - Easy** depth 2, simple evaluation weights (distance 3, walls 1, mobility 1), no A*, only pawn moves (walls never placed).
 - Medium** depth 3, higher weights for distance and mobility, still no walls.
 - Hard** depth 7, A* for precise distance estimation, and the AI actively considers wall placements using the heuristic described above.
7. Transposition table: A global dictionary is used to cache minimax evaluations. The transposition table is cleared at the beginning of each AI turn to avoid stale data from previous moves or games
8. No sound: The game is completely silent; no audio feedback is provided.
9. Input blocking during AI turn: The UI completely ignores mouse clicks and mode-switching keys while the AI is "thinking". This prevents race conditions and simplifies the turn management.
10. AI move delay: A fixed delay of 500 ms is applied before executing the AI's move. This is purely for visual clarity and does not affect the AI's search time (the search runs instantly, then the program waits).
11. Pygame initialisation: The game assumes Pygame can be imported and that a display surface can be created with the dimensions defined in constants.py.

These assumptions align with the project requirements and the game rules, thus allowing a complete, playable Quoridor implementation.

4 Implementation Challenges

4.1 AI performance

The Hard AI performs a depth-7 minimax search, which can explore thousands of states. Evaluating each state with a full BFS or A* for distance would be too slow. Additionally, the game must remain responsive during the AI's turn.

Solution:

Transposition table We implemented a Zobrist-like hasher (`ai/hasher.py`) that hashes the board state (player positions and set of placed walls). The global `Transposition_Table` dictionary caches (depth, value) pairs. When the same state is reached at a depth equal or greater, the stored value is reused, cutting search time dramatically.

Move ordering Before exploring pawn moves, we sort them using a cheap heuristic (closeness to goal and opponent's distance). This improves alpha-beta pruning effectiveness, where in tests, it reduced the number of evaluated nodes by about 40%.

A* vs BFS For Hard difficulty we use A* with a simple Manhattan heuristic, which is faster than BFS for distance estimation. Easy & Medium use BFS for simplicity.

Delayed AI move In the main loop, when it is the AI's turn we store a timestamp and only execute `do_ai_move()` after 500 ms have passed. This gives the player visual feedback and prevents the AI from responding instantly, which feels unnatural.

4.2 AI wall placement blocking itself

The AI should place walls that hinder the opponent but never block its own path. Designing a heuristic that balances both goals without exhaustive search was difficult.

Solution:

- We generate candidate walls only in a 3×3 neighbourhood around each player's pawn. This limits the search space to about 60 walls per turn.
- For each candidate, we temporarily add the wall to a deep copy of the board and compute the change in A* distance for both players using the formula:

$$score = 2 * (human_new_dist - human_old_dist) + (ai_old_dist - ai_new_dist).$$

Where the factor of 2 prioritises blocking the human.

- If a wall would make either player's distance infinite (unreachable goal), the score is set to a very large negative number, effectively rejecting it.
- The best wall score is compared with the best pawn-move score (from minimax). The AI chooses whichever action yields a higher score. This gives Hard AI a strategic option to place walls when beneficial.

5 AI Algorithm Explanation

The computer opponent in our Quoridor implementation uses a game-theoretic search algorithm combined with heuristic evaluation. The AI is designed to work at three difficulty levels (Easy, Medium, Hard), which differ in search depth, evaluation weights, and the ability to place walls.

5.1 Core search: Minimax with Alpha-Beta pruning

The AI treats Quoridor as a zero-sum, perfect-information, turn-based game. It uses the **minimax** algorithm with **alpha-beta pruning** to decide the best move. The state is represented by the board (walls), the positions of both pawns, and the number of walls each player still holds.

The recursive function `minimax(board, human, ai, maximise, depth, alpha, beta, weights, use_astar)` returns a numerical evaluation of the state. When `maximize` is true, it is the AI's turn; otherwise, it is the human's turn. The search stops when:

- The AI reaches its goal row (return $+\infty$).
- The human reaches its goal row (return $-\infty$).
- The remaining depth is zero – a heuristic evaluation is used.

Alpha-beta pruning cuts off branches that cannot influence the final decision, significantly reducing the search tree.

5.2 Evaluation function

The evaluation function computes a score from three weighted components:

$$\text{score} = w_d \cdot (d_{\text{human}} - d_{\text{ai}}) + w_w \cdot (w_{\text{ai}} - w_{\text{human}}) + w_m \cdot (m_{\text{ai}} - m_{\text{human}})$$

where:

- d_{player} = shortest path distance from the pawn to its goal row (computed with BFS or A*).
- w_{player} = number of walls the player still possesses.
- m_{player} = number of legal pawn moves (mobility).

The AI tries to minimise its own distance while maximising the opponent's distance, keep a wall advantage, and increase its own mobility relative to the opponent.

5.3 Move ordering for better pruning

To make alpha-beta pruning more effective, the AI sorts the legal pawn moves before exploring them. The sorting heuristic gives higher scores to moves that:

- Bring the pawn closer to the goal (low d_{ai}).
- Increase the opponent's distance (high d_{human}) – e.g., blocking a central path.

The sorted list is then searched in order from “most promising” to “least promising”.

5.4 Transposition table

To avoid re-evaluating identical states reached through different move sequences, a **transposition table** (hash map) is used. The board and player positions are hashed with a Zobrist-like function. For each visited state we store the search depth and the computed value. If the same state is reached at the same or higher depth, the cached value is returned directly.

5.5 Pathfinding for distance estimation

Two pathfinding methods are implemented:

- **BFS (Breadth-First Search)**: Guarantees the shortest number of orthogonal steps respecting walls. It is used for implementing Easy & Medium difficulties and validating wall placements. Complexity $O(\text{grid size})$.
- **A***: Uses the Manhattan distance to the goal row as a heuristic. It is more efficient on large boards and is enabled only for Hard difficulty to obtain more accurate distance estimates.

The distance values are essential both for the evaluation function and for the wall placement heuristic.

5.6 Wall placement strategy

For Hard difficulty only, the AI also considers placing a wall instead of moving its pawn. The algorithm proceeds as follows:

1. Generate candidate walls near the AI's own pawn and near the human's pawn (within a 3×3 neighbourhood). Each candidate is checked for legality (bounds, no overlap, no crossing, no complete blockage).
2. For each legal candidate, temporarily place the wall and compute:

$$\text{wall_score} = 2 \cdot (d_{\text{human}}^{\text{new}} - d_{\text{human}}^{\text{old}}) + (d_{\text{ai}}^{\text{old}} - d_{\text{ai}}^{\text{new}})$$

This heuristic rewards walls that increase the human's distance a lot (multiplied by 2) while not harming the AI's own path.

3. The best wall score is compared with the best pawn-move score obtained from minimax. If the wall score is higher, the AI places the wall; otherwise it moves its pawn.

5.7 Difficulty levels configuration

The three difficulty levels are defined in `ai/config.py` as follows:

Difficulty	Depth	Weights (w_d, w_w, w_m)	Pathfinding	Uses walls?
Easy	2	(3, 1, 1)	BFS	No
Medium	3	(10, 1, 10)	BFS	No
Hard	7	(15, 10, 5)	A*	Yes

Table 1: AI Difficulty Configuration Parameters.

Easy and Medium never place walls; they only move the pawn. Hard difficulty performs a deep search, uses precise A* distances, and actively places walls to trap the opponent.

5.8 AI turn implementation

During a human-vs-AI game, the AI controls Player 2. When it is the AI's turn, the main loop waits 500 ms (for visual feedback) and then calls `do_ai_move()`. This function:

1. Retrieves the appropriate difficulty configuration.
2. Calls `get_best_move()` from the AI controller.
3. The controller returns either `['move', (r,c)]` or `['wall', wall]`.
4. The move is executed via the same game methods used for human players, ensuring rule consistency.

All user input is blocked during the AI's "thinking" phase to prevent race conditions.

5.9 Performance considerations

- The transposition table reduces redundant search, especially at depth 7.
- Move ordering improves pruning efficiency; testing showed a reduction of explored nodes by approximately 40%.
- Candidate wall generation is limited to a 3×3 area around both pawns, keeping wall evaluation tractable (at most ~ 60 candidates per turn).
- A* with a simple heuristic runs quickly on a 9×9 grid (maximum 81 nodes).

These choices allow the Hard AI to make a move within a few seconds on modern hardware, while Easy and Medium respond instantly.

References

- [1] Gigamic, “Quoridor – Modern Classics,” *Gigamic*. [Online]. Available: <https://en.gigamic.com/modern-classics/107-quoridor.html>
- [2] “Tic Tac Toe: Understanding the Minimax Algorithm,” *Never Stop Building*. [Online]. Available: <http://neverstopbuilding.com/minimax>
- [3] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2009.
- [4] A. Patel, “Introduction to the A* Algorithm,” *Red Blob Games*. [Online]. Available: <https://www.redblobgames.com/pathfinding/a-star/introduction.html>
- [5] “Quoridor Gameplay and Strategy Tips,” *YouTube*. [Online]. Available: <https://www.youtube.com/watch?v=6ISruhN0Hc0>